Group 15 assessment and oral exam preparation

Group 15 • Final assessment preparation • 5 November 2026

This pack explains the temperature observability implementation, why its architecture was chosen, what the measurements establish, and how all three members can defend it. It targets the Excellent descriptors in the official rubric through working behavior, consistent data, strict notification deadlines, variable HTTP validation, and evidence of different bottlenecks.

Status on 5 November 2026: the deployment has been recovered. A fresh read-only check found healthy API and dashboard endpoints, 16 assigned partitions on `experiment`, one executor with capacity eight, no queued application work, and stopped local workload generators. The earlier broker check reached its sampled high watermarks. These are dated observations, not a guarantee of future health. Fresh readiness, broker evidence.

The latest application suite passed **92 tests**, including 14 integration tests. The original instructor tools passed a variable HTTP run and a fixed 200-request/second target run. The independent live-history 200-request/second profile still failed; the 25/100/50 profile passed. Outage-era notifications were late. These distinctions are essential to the exam defense.

Documents and reading order

Document	Purpose
Submission report	A coherent account of requirements, design, implementation, evaluation, and limitations
Rubric and evidence	Map the actual grading descriptors to implemented behavior and demonstrable evidence
Architecture and correctness	Understand ownership, alternatives, transactions, caching, ordering, and scaling
Performance and development	Explain the measurements, failures, revisions, and unresolved capacity limits
Oral exam handbook	Practise 60 questions, follow-ups, and worked exercises
Team rehearsal	Cross-train the Computing Science, Data Science, and Business Informatics members
Demo runbook	Verify the current mode and panels without accidentally starting local workload
Quick reference	Review the essential facts immediately before the exam
Code map	Locate every implementation responsibility, test, and complete code appendix

Read the combined PDF, print individual reports from pdf/, or edit the Markdown sources. The submission PDF is separate from the internal rehearsal material. Adapt it to any separately specified course page limit; the supplied rubric gives no report-length rule.

How to use the pack

Start together with the submission report and quick reference. Each person should then draw the architecture, trace one complete sensor group, explain two crash windows, interpret one passing and one failing workload, and diagnose two different bottlenecks. Use the oral handbook with random redirection rather than allocating exclusive knowledge to a specialist.

Suggested preparation leads are Computing Science for correctness/concurrency, Data Science for measurement/experimental reasoning, and Business Informatics for requirements/value/cost/operations.

These are study roles; they do not claim who authored the implementation. The team plan includes an unfilled contribution record for truthful attribution.

The most effective answer structure is **decision, mechanism, evidence, limitation**. For example: we separate delivery from ingestion; a durable outbox joins alert obligations to the history transaction; recovery tests preserve three expected alerts; an external acceptance followed by a lost acknowledgement can still produce a retry.

Evidence and reproducibility

The source manifest identifies the current application, test, runner, and deployment files. The current code appendix includes their full contents. The original implementation report and November change report explain the development changes and retain their complete patches.

All numerical workload claims refer to saved benchmark artifacts. Capacity tests preceded the final per-partition backpressure change; that change passed the final suite and was exercised during real backlog recovery. We do not relabel the older capacity tests as a fresh performance test of that final scheduler revision.

The earlier November 3 pack remains an archive. Use this pack for current readiness, test totals, caching, cost calculation, and operational guidance.

The report builder only reads source/evidence and writes documents. It does not operate Docker or generate traffic. Install the documentation dependencies in a separate environment, then run:

```
python reports/exam-2026-11-05/build_reports.py
python reports/exam-2026-11-05/verify_reports.py
```

Verification checks evidence values, source hashes, links, PDF text/layout, and private-value exclusion. The portable assessment ZIP preserves repository-relative paths and includes the source and evidence needed by these reports. It excludes runtime credentials and private environment files.

Rubric and evidence for the final assessment

Group 15 • Final assessment preparation • 5 November 2026

The official rubric, page 1, allocates **70% to the final demo and 30% to architecture**. A working demo is a prerequisite. Excellent requires both successful variable-load behavior and an explanation of different architectural bottlenecks under load. A report supports that demonstration; it cannot substitute for it.

What the grading bands require

Component	Fair	Good	Excellent
Demo, 70%	Working interfaces and consistent data satisfying requirements	Also passes variable workload stress with notification latency below 10 seconds	Also keeps up with variable HTTP workload
Architecture, 30%	Interfaces, consistency, and separation of concerns	Distinct responsibilities separated with scalability in mind	Also pinpoints different bottlenecks when load is applied

These are paraphrases, not invented numerical grading rules. The rubric gives no numeric conversion from these bands. It states that individual grades can differ based on contribution and knowledge and that questions may be redirected to any group member.

The ordinary final assessment is assumed here. The rubric's special note limits an additional discretionary resit beyond the midterm/final opportunities to a passing grade for both demo and architecture. If that situation applies, stronger reports cannot remove the policy ceiling.

Requirement to implementation to evidence

Requirement or criterion	Implemented mechanism	Evidence to show and its scope
Observe temperature; do not control it	Pure sensor-group state machine; no actuator interface	`engine.py`; worked exercise in the handbook
Query from configuration onward	Configuration persisted before acknowledgement; metadata endpoint; empty history is valid	API and real-database integration tests
Correct temperature and lifecycle	All allocated sensors for one measurement; inclusive range; explicit phase transitions	Engine tests; original instructor ground truth
Required historical queries	Ordered carry-out arrays, inclusive time bounds, all outside points	202 complete endpoint comparisons in each final native run
Correct transition notifications	Stabilized on entry/re-entry during stabilization; one OutOfRange per running excursion	Notification sets checked against original producer truth
Strict latency	Every required accepted alert must have latency below 10 seconds	Native variable: 703/703, maximum 2.884 s; native fixed: 596/596, maximum 8.958 s
Variable HTTP behavior	Original Rust variable generator plus per-phase validation; independent open-loop profiles	Five native targets passed; independent 25/100/50 passed; independent 200 peak failed
Distinct scaling responsibilities	Ordered scheduler, replaceable executors, authoritative Data, gateway, durable delivery	Architecture diagram, source walkthrough, replica experiment
Pinpoint bottlenecks	Pool wait/occupancy, CPU, queue/partition, outbox, cache, disk metrics	State-write overhead, repeated history work, controller CPU, and shared-host saturation observations
Load-based scaling	Host-side bounded scaling policy; worker discovery and capacity reservations	Policy and scheduler tests; policy is available but not continuously enabled in the demo
Cost overview	One VM plus provisioned storage estimate; Prometheus time series	Configured estimate with gp3 assumption and explicit exclusions
Recoverability	Durable partial groups, checkpoints, outbox, preserved named volumes	Recovery test; recovered database; retained Kafka backlog catch-up

Primary artifacts: final tests, native variable, native fixed, independent passing, independent boundary failure, recovery.

What supports Excellent now

We can demonstrate the required interfaces and consistency, explain separate scaling/failure responsibilities, present a genuine changing-HTTP workload, and identify multiple limiting stages using measurements. Strict boundary tests reject exactly ten seconds. The latest test suite and the live recovery evidence support the final scheduler revision.

The native variable run used 100 simultaneous main experiments, ten sensors each, one-second samples, and an HTTP generator configured to choose targets from 100 through 200. The observed targets were 148, 182, 181, 114, and 106. Each settled phase passed our validation. This does not mean that the random run actually included a 200 phase; the separate fixed run supplies that evidence.

Our local HTTP acceptance rule is at least 95% of the target, with correctness/error checks. **The rubric does not specify 95%.** Native 200 achieved 190.573/s, only 0.573/s above that local threshold. Say this clearly when asked about headroom.

The native producer runs a configured concurrency and sample period with lifecycle changes. These results do not prove every possible variable event-arrival schedule. The independent HTTP test queries live growing histories and measures offered-load omissions/draining, which makes its failed 200 peak relevant even though the native tool passed.

Remaining work with the highest assessment value

Priority	Action for the team	Completion evidence
1	Keep the actual demo healthy and verify external API reachability before assessment	Fresh mode/health/assignment/outbox checks and a laptop/public-route query
2	Rehearse all members across the whole design	Every member explains invariants, one code function, two bottlenecks, and a failed test
3	In a separate test window, repeat representative workloads on the final scheduler revision	Repeated archived runs with exact source/image identity and all outcomes retained
4	Characterize capacity instead of claiming it is unlimited	Longer histories, cold caches, event-rate steps, skew, and soak tests with queue drain
5	Address the measured limiting resource if deployment constraints allow	Controlled comparison on resized/separated infrastructure, including cost

These are follow-up experiments and preparation tasks, not completed tests hidden behind a checklist. No new stress workload was started while preparing these reports.

Claims to avoid and the correct replacement

Overstatement	Defensible statement
All green means the professor passed us	Our added local status panel records our checks on a specified local run; hosted grading needs its own current evidence
The system sustains any 200-rps workload	The native fixed target passed narrowly; the independent live-history 200 peak failed
Microservices make this highly available	Responsibilities are separated, but all services share one VM and disk
Exactly-once notifications	Durable deduplicated database obligations, with external delivery uniqueness requiring receiver idempotency
All outage data was restored	Named-volume data reopened and retained Kafka backlog was processed; broker-expired input cannot be reconstructed from these checks
The scheduler change caused the measured capacity gain	The capacity measurements preceded that change; its separate evidence is tests and backlog recovery
Our cost graph is the AWS bill	It is an estimate using one VM and provisioned disk, with an unconfirmed disk type and stated exclusions

The strongest defense is precise: explain what exists, what a particular experiment measured, what failed, and what you would test next.

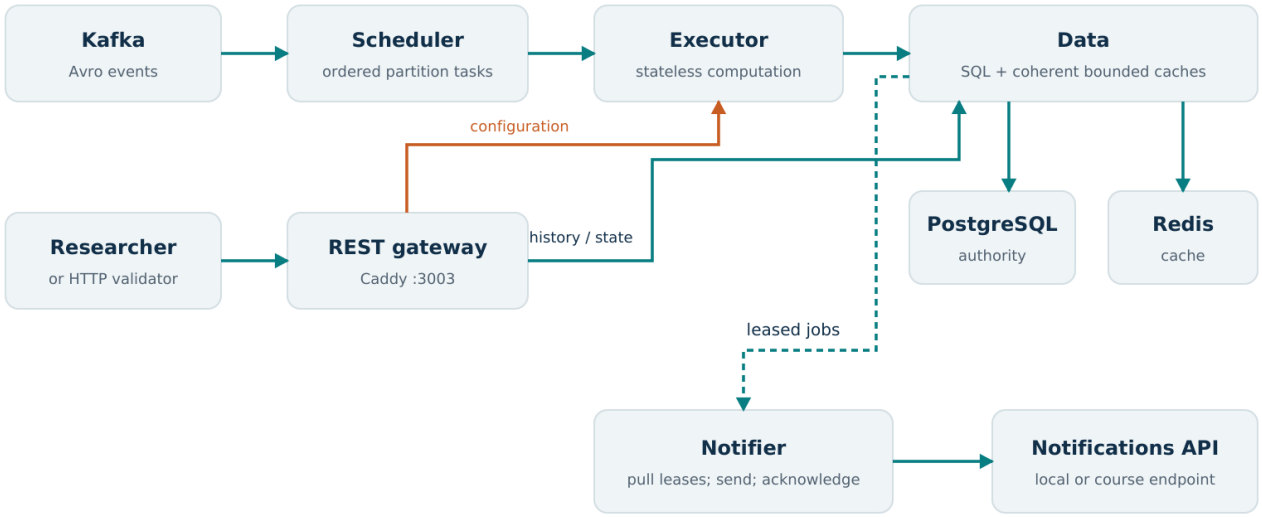
Architecture and correctness defense

Group 15 • Final assessment preparation • 5 November 2026

The central design decision is to commit experiment state, measurements, notification obligations, and processing progress together before acknowledging Kafka. Service boundaries then separate responsibilities that have different ordering, latency, and scaling requirements. Every optimization must preserve that consistency boundary.

Responsibilities and deployment

Application responsibilities and data flow



PostgreSQL owns durable state. Redis is disposable. The notifier pulls jobs from Data.

Component	Owns	Scaling or failure boundary
Kafka consumer and scheduler	Avro intake, partition queues, batch order, reservations, offset acknowledgement	One active task per partition; hot keys and broker retention
Stateless executor	State-machine computation and commit proposals	Capacity eight per replica; shared Data and CPU remain downstream
Data	Durable writes, history reads, cache coherence, notification leases	One process owns writes and local invalidation; PostgreSQL capacity
Caddy public API	Public routing and transport	Downstream availability; extra gateway overhead; public routes only
Notification dispatcher	HTTP delivery, retry, lease acknowledgement	Receiver latency, concurrency, lease recovery
PostgreSQL	Authoritative state, history, checkpoints, jobs, rejected input	Disk/CPU/locks/connections; single deployment
Redis	Disposable serialized history responses	Bounded memory and hit rate; fallback increases database work

Prometheus, node exporter, and Grafana observe the system. They do not participate in the experiment transaction. The local instructor producer, HTTP validator, notification validator, and their truth database are test infrastructure, distinct from the application under test. All are cohosted in local capacity tests.

The public port is 3003. Caddy forwards history/state reads to Data and configuration requests to an executor. The Python `api.py` remains the development API/OpenAPI definition, while the deployed public request path uses Caddy. Internal load/commit/claim endpoints are not exposed by that gateway.

One measurement through the complete system

Suppose experiment E has sensors A, B, and C. They report measurement M at time t, with a shared opaque measurement hash. A reading from A does not establish the mean. The executor stores it in a partial group and commits that state. A different executor can later load the group, add B and C, and complete the calculation.

The engine validates allocation, matching identity/timestamp/hash, duplicate consistency, phase, and temporal order. Once complete, it uses `math.fsum(values) / count`, classifies the inclusive range, updates prior range state, and creates a measurement plus any transition notification. Later complete groups wait behind earlier incomplete groups within an experiment.

The executor loads a database snapshot, processes a bounded batch, and posts a commit proposal. Data checks the expected checkpoint and experiment versions. Within one transaction it writes state, measurements, jobs, rejected events, and progress. A successful response lets the scheduler commit Kafka's next offset. The notifier subsequently leases an outbox job and sends the required payload; the original opaque hash is forwarded as `cipher_data`, not decrypted by our implementation.

State machine and interface semantics

State or transition	Behavior
Configured	Configuration persisted; state queryable; required histories may be empty
Stabilizing	Complete sensor groups monitored; Stabilized on entry/re-entry into the allowed range
Started	Researcher event enters running; prior excursion state resets
Running	Every complete average retained; OutOfRange only when starting an outside excursion
Terminated	Retains history and termination time; new measurements rejected

The engine also permits direct start from configured and cancellation before carry-out. A transition with incomplete groups is rejected rather than fabricating missing temperatures. Duplicate identical configuration and lifecycle events are handled idempotently; conflicting configuration is rejected.

Temperature thresholds are inclusive: a mean equal to either boundary is in range. History start/end bounds are inclusive, with stable ordering for equal timestamps. The required outside endpoint includes every outside carry-out point. The interval convenience endpoint closes an excursion at the next in-range point or termination time; an ongoing excursion has an open end.

Stabilization measurements are stored internally but filtered out of required carry-out histories. This supports completed-ID deduplication without changing the external query contract. Engine, schema, history rendering.

Ordering, replay, and concurrent writes

The scheduler constructs a stream identity from consumer group and topic and permits one task at a time per assigned partition. It processes Avro object-container records and treats the `record_name` Kafka header as authoritative. This accommodates the supplied producer's termination-schema mismatch without modifying instructor code.

Partition order depends on the producer consistently routing one experiment to one partition. Adding partitions or changing key routing during an experiment is therefore not a harmless capacity switch. Versions can detect competing writes, but they do not invent a missing total order between different input

partitions.

Kafka offset commits and PostgreSQL transactions are separate operations. The application checkpoint links database effects to input progress. If Data commits offset 45 but Kafka still points to 41, replay loads the durable checkpoint and suppresses already committed effects; Kafka can then commit next-read offset 46. A duplicate completed measurement at a new Kafka offset is separately detected by its logical identity.

Batch state writes sort experiment IDs, pipeline adjacent inserts/updates, and check every returned version. A stale version fails the transaction, so the executor reloads and recomputes; it does not publish a partially updated batch. The integration test with a stale middle version is stronger than merely testing that a loop invokes a database method.

The read path and its correctness

Three layers have different purposes. The metadata cache avoids repeated version lookups. The bounded snapshot cache reuses a whole small carry-out history across different query windows. Redis stores serialized responses for exact version/query combinations; indexed SQL serves misses and larger histories.

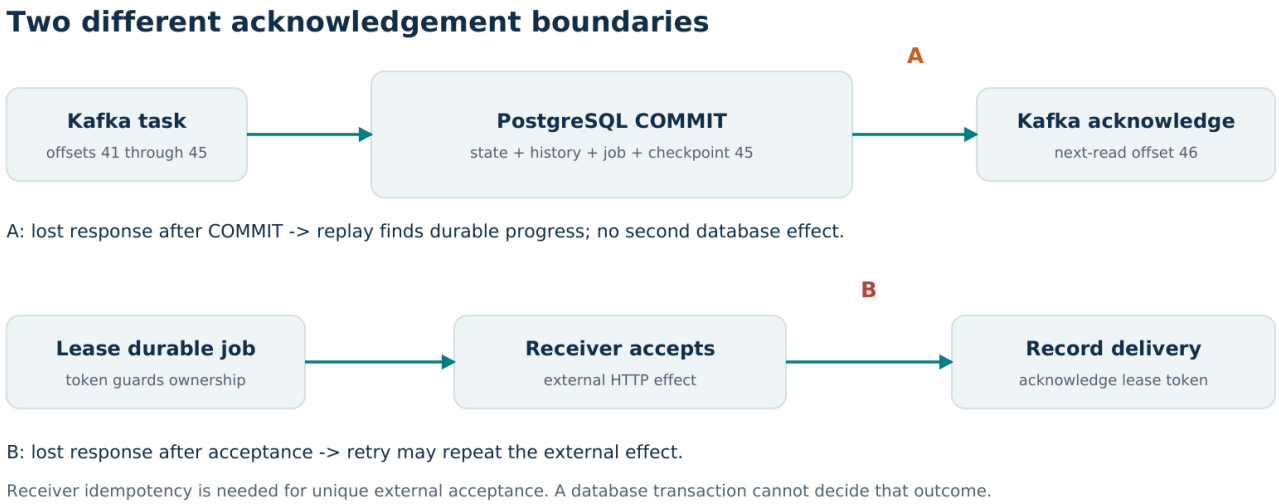
A bounded probe reads metadata/version and at most 513 points in one SQL statement. Histories of at most 512 points can become a snapshot. Large histories receive an eviction-bounded marker and fall through to the original query path; they are not truncated to 512 points. The cache allows at most 16,384 points and 256 entries, including empty histories and large markers. `TO_HISTORY_CACHE_POINTS=0` disables it for comparison.

Binary search locates inclusive time boundaries. A cached snapshot must have a version at least as new as the request's metadata requirement. An older fill cannot replace a newer cached snapshot. Concurrent cold readers share a per-experiment fill lock; locks are released and removed when no waiter remains.

Metadata is invalidated around writes. Commit-time warming uses values returned/known by the successful transaction, but only when its generation remains the final uncontended writer. The important race is: writer A commits, releases its database connection slowly, writer B commits, then A resumes. Without a generation fence, A could replace newer metadata. The regression test forces this interleaving.

The guarantee is that an acknowledged write is not followed by indefinitely obsolete cache state. Concurrent reads may observe an earlier committed snapshot according to when they started. We do not claim a distributed linearizable cache across multiple Data processes. The current invalidation protocol assumes one Data process owns all writes. Data, integration tests.

Durable delivery and failure windows



Failure	Result and recovery
Before the database commit	Proposed effects roll back; input can be recomputed
After commit, before Kafka acknowledgement	Durable checkpoint/history/outbox exist; replay is deduplicated
Worker disappears with a partial sensor group	Persisted partial state is reloaded by a replacement
Redis unavailable	Bounded cache failure/bypass; authoritative SQL fallback
Notifier stops after claiming	Lease expiry permits another claim
Receiver accepts but acknowledgement is lost	Retry can repeat external acceptance; receiver idempotency is required
Permanent input error	Durable quarantine permits partition progress; invalid experiment input is not repaired
Whole VM/disk unavailable	All colocated services may stop; container boundaries do not provide redundancy

The notifier polls after an empty/failed claim every 0.1 seconds, sends up to 32 jobs concurrently, uses a two-second request timeout, and normally drains a nonempty backlog immediately. The lease is fifteen seconds. Results acknowledge a lease token so a stale worker cannot overwrite a newer lease holder's result. Retries use bounded delays; selected permanent HTTP failures are marked failed for inspection.

Reducing the poll interval alone cannot remove sensor-group waiting, Kafka queueing, SQL delays, receiver latency, or lease recovery. At a healthy empty-poll interval of 0.1 seconds, polling is only one part of the latency budget. A crash recovery lease can independently exceed the ten-second target.

Backpressure and scaling

The global scheduler guard pauses at 8,192 queued plus in-flight events and resumes below 4,096. A fetched batch can overshoot a threshold; these are control watermarks, not a byte-exact memory bound. The Kafka client has its own buffering, and a task batch defaults to 256 events.

The final revision adds per-partition control. With 16 assigned partitions, the default share is max(256, 8192/16) = 512 events. A partition pauses at that share and resumes below half, subject to the global guard. Counting in-flight work prevents dispatch from appearing to free capacity before the batch completes. Assignment/revocation clears pause bookkeeping. Pausing leaves waiting input in Kafka; it does not guarantee deadlines during overload.

Executor discovery samples worker load and uses local reservations. Effective load is the greater of reported active work and reserved work. Least-busy dispatch is appropriate for broadly similar batches, but sensor counts, batch composition, and query/write costs can differ. Worker capacity enforcement remains the final guard.

The host-side autoscaler defaults to one through four replicas. It considers queue above 1,024 or occupancy at least 75% busy, scales up after five seconds, and scales down after sixty seconds fully idle. It is opt-in, not running continuously in the demo. The policy observes executor demand, not all downstream bottlenecks; extra executors on a CPU-bound host can make HTTP service worse.

For E experiments, S sensors, period T, input rate is E×S/T and aggregate rate E/T. For P partitions, K replicas and capacity C, active partition tasks are bounded by min(P, K×C). Service time, skew, database contention, and CPU determine actual throughput. These formulas should not be presented as measured capacity.

Alternatives and tradeoffs

Choice	Why it fits this implementation	Cost or alternative
Explicit services	Isolate ordering, computation, queries, and external delivery	A modular monolith could reduce hops/operations on one VM
Python with asynchronous I/O	Pure domain code, rapid validation, concurrent network/database work; measured workload passes	CPU-heavy work remains constrained; language alone does not guarantee performance
PostgreSQL authority	Atomic cross-table effects, indexes, constraints, locking and versions	Shared database/process can bottleneck; replication needs a consistency design
Redis plus bounded snapshots	Reuse exact responses and different windows over small histories	Coherence complexity; small-history gains do not extrapolate to arbitrary duration
One measured executor	Smallest passing configuration in the earlier replica study	Requires reevaluation on a different workload or host
Durable outbox	Survives process loss and separates receiver delays	External duplicate window; lease/retry delay
Backpressure	Bounds application intake and improves fairness	Waiting moves upstream; lag/age must be observed

How to identify different bottlenecks

Observation	First hypothesis	Discriminating check and response
High CPU, growing queue and HTTP latency	Shared host saturation	Separate generator/service CPU; reduce proven overhead or add allowed resources
Long pool wait but high connection occupancy	Data/database contention	Compare operations and SQL/lock work; increasing the pool can worsen pressure
HTTP slows on cold/long histories while ingestion is healthy	Read/serialization workload	Compare cache hit/miss, history size, SQL access and response bytes
Free workers but uneven partition queues/lag	Key skew or sequential work	Inspect partition progress; fairness helps other partitions but cannot parallelize the hot one
Current history but aging outbox and retries	External receiver/network	Inspect outcomes and receiver delay; executor replicas are the wrong first action
Disk shrinking despite low CPU	Persistent history or disposable storage growth	Attribute volume/log/cache growth; preserve data and plan capacity

Pool occupancy includes transaction execution, application work, and scheduling while a connection is held; it is not isolated SQL duration. Scheduler queue is not total broker lag. A p99 latency panel does not identify a missing alert. These distinctions keep the diagnosis tied to what each metric actually measures.

The performance report separates observed bottlenecks from hypotheses. The code map links each mechanism to its implementation and tests.

Performance measurements and development history

Group 15 • Final assessment preparation • 5 November 2026

The project improved through failures that could be linked to specific work: sequential state writes, repeated history/metadata reads, expensive monitoring counts, and test-controller overhead. The final results support the original tools' selected workloads while retaining an independently measured capacity limit. They do not establish unlimited throughput or a hosted professor-test pass.

Environment and measurement meaning

The host record identifies a t3a.medium in eu-west-1, two vCPUs, about 3.754 GiB memory, and 20 GiB provisioned disk. Application, local Kafka, databases, monitoring, and generators share the host in local tests. CPU saturation therefore describes the whole test setup, not solely application code. VM burst-credit behavior and longer-duration performance were not separately characterized.

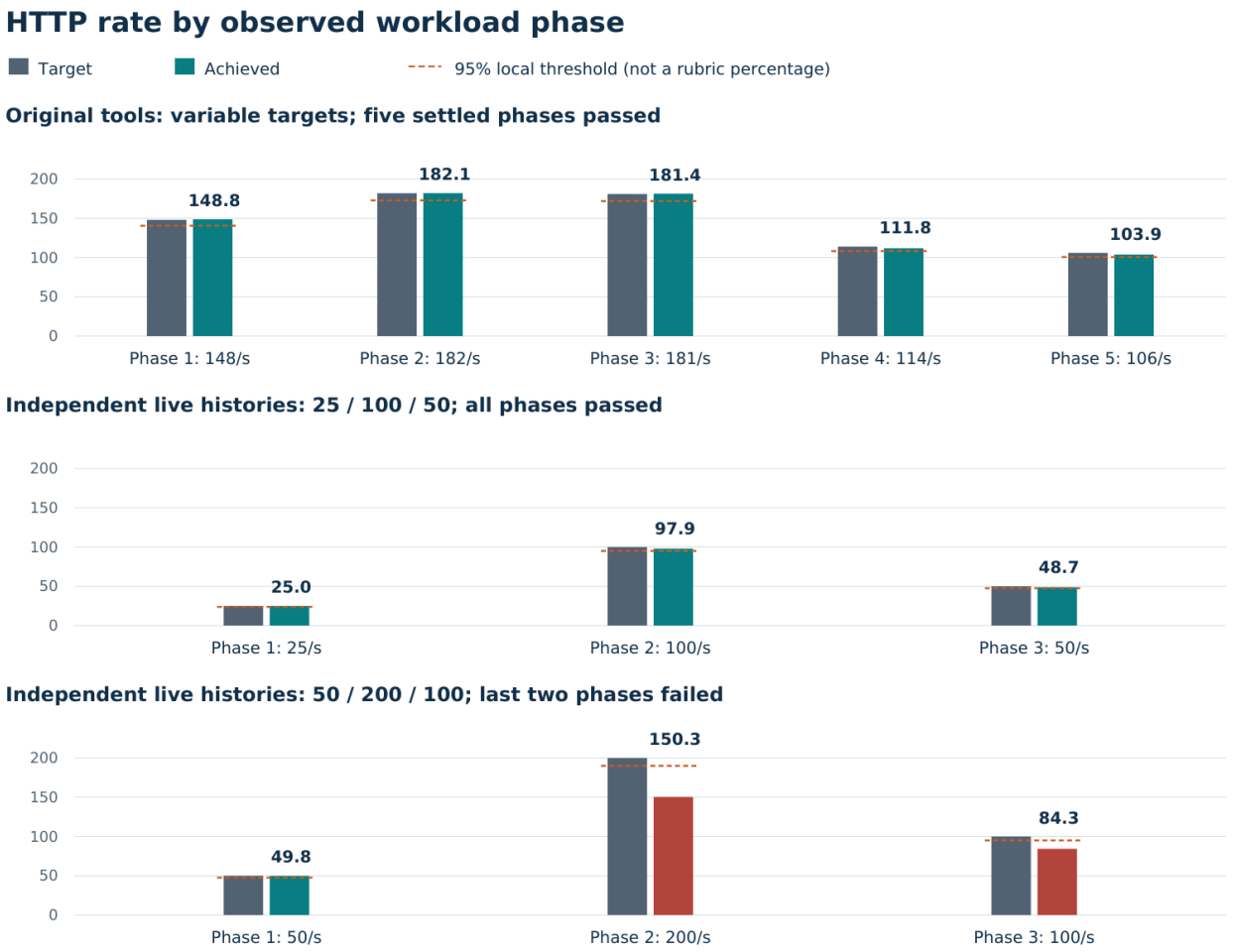
The native harness executes the assignment's original Rust producer, notification validator, and HTTP validator. Their logic was not edited. It stores original ground truth and adds explicit nonzero-workload, complete-history, strict-latency, and throughput checks. The independent Python harness constructs expected histories/alerts and uses a local notification sink; it tests a different workload path.

Native HTTP batches are closed-loop: the effective gauge describes the last completed batch, while the target gauge describes current work. Variable validation groups observed targets, excludes the first two seconds after transitions, requires at least five seconds of settled evidence per counted phase, and requires at least two distinct targets. It compares both completed counter rate and settled effective rate with 95% of target. These exclusions mean transition behavior needs separate examination; the independent test includes scheduling and drain effects.

Independent phases start from intended send time. Latency includes generator delay and semaphore waiting. Completion after a phase extends that phase's rate denominator. Pending tasks are capped at 1,000; an omitted offer fails the test. A later easy phase cannot average away a failed busy phase.

Event-to-acceptance latency also depends on comparable producer and receiver clocks. The local runs share the host clock; the external path needs clock synchronization to interpret timestamps correctly. Our notifier clamps negative elapsed values to zero, so its metric alone cannot diagnose clock skew. A wall-clock adjustment is a different issue from queueing delay.

Latest capacity results



Original-tool settled phases exclude transition seconds. Independent rates include phase drain and detect omitted offers.

Different workloads and query histories; these plots are not a paired comparison. Saved measurements: 5 November 2026.

Workload	Required and received alerts; maximum	HTTP result
Native variable 100–200 configured, 80 main samples	703/703; 2.883569 s	Five observed phases passed; zero validation errors
Native fixed 200/s, 60 main samples	596/596; 8.957969 s	190.573087 achieved/s; local pass with little headroom
Independent 25:15,100:30,50:15	1,626/1,626; 6.189247 s	Achieved 25.000, 97.926, 48.744/s; all phases pass
Independent 50:15,200:30,100:15	1,626/1,626; 8.526757 s	Achieved 49.821, 150.329, 84.335/s; final two phases fail

Both native runs used 100 concurrent main experiments, ten sensors, a one-second sample period, one completed warmup, and at least ninety seconds of HTTP observation. Each checked both complete history endpoints for 101 experiments: **202 comparisons**, not 202 experiments. Random temperatures explain different expected alert counts.

The native variable sequence was **148, 182, 181, 114, 106** requests/second. Settled achieved rates were 148.770, 182.099, 181.406, 111.826, and 103.876. The configured upper bound 200 was not itself observed in that random sequence. The fixed run addresses that target separately.

The fixed native result exceeds the local 190/s threshold by only 0.573/s, with a 1.042-second observed notification margin below ten seconds. These margins are results of one short run. The rubric says keep up

with HTTP; it does not define our 95% rule or promise that the professor uses it.

The failed independent busy phase intended 6,000 requests but scheduled only 5,526. All scheduled requests succeeded, yet 474 offers were omitted and completion took 36.759 seconds from phase start, producing 150.329/s. Zero HTTP errors is therefore compatible with a failed throughput requirement. Its busy-phase HTTP p95 was 7.189 seconds; this statistic must not be assigned to the native fixed run.

Sources: native variable, native fixed, independent pass, independent failure.

What changed during development

Observation	Change and protected invariant	Measured result or practical lesson
Separately arriving readings	Persist partial groups; group by measurement identity and allocated sensors	Replacement workers preserve completeness; no mixed-time averages
Termination schema/header mismatch upstream	Honor record_name header; add codec regression	Compatibility without changing instructor tools
External Kafka connection failures	Diagnose public-name certificate mismatch; apply demo-specific hostname exception while retaining CA/client authentication	Fresh preflight obtained 16 assignments; ports alone had not proved Kafka readiness
Published test image/configuration mismatch	Build original Rust sources with local test credentials	Independent original-tool validation became reproducible
October native stress: three late alerts and 86.848 achieved/s	Pipeline sorted version-checked state writes; test stale-middle rollback	Next run: 5.641-s maximum and 97.310 achieved/s, with correct histories
More executors reduced combined HTTP throughput	Select one measured passing replica, keep bounded scaling available	Shared resources matter more than container count
November recovered DB had millions of measurements	Reduce historical metric scan frequency; add status index	Avoid repeated scans over a 125 MB outbox heap
Repeated metadata/history round trips	Warm committed metadata with generation fence; bounded small-history snapshots	Independent 200 peak improved, but remained a failure
Native variable phases still lagged	Replace repeated Compose resolution with cached IDs and one inspect call	Later native variable run passed; controller cost is part of the shared setup
Raw transport/uvloop trial underperformed	Revert the trial; keep its failed evidence	Fewer abstractions did not establish a better end-to-end result
Retained backlog exposed hot partitions	Add per-partition watermarks with global guard, preserve ordering	Final tests and real backlog exercise; no isolated healthy-load speedup claim
Disk exhaustion removed runtime availability	Recover named volumes; retain compiled tool image; source/image cache reuse; log/retention limits and disk alerts	Approximately five GiB reclaimed headroom before report dependencies; authoritative volumes retained

These are recorded development observations. They do not establish that every feature was authored on its first mention or that every team member personally implemented it.

Before and after evidence

October's initial native run delivered 632/632 alerts, maximum 10.715822 seconds, and achieved 86.847621 HTTP/s against 100 offered. The following run delivered 610/610, maximum 5.640533 seconds, and achieved 97.309762/s. Both had 202 correct history comparisons. The observed changes are 47.36% lower maximum delay and 12.05% higher achieved rate, not isolated or replicated causal estimates.

November's independent 200-peak sequence retained the same configured profile and expected notification count:

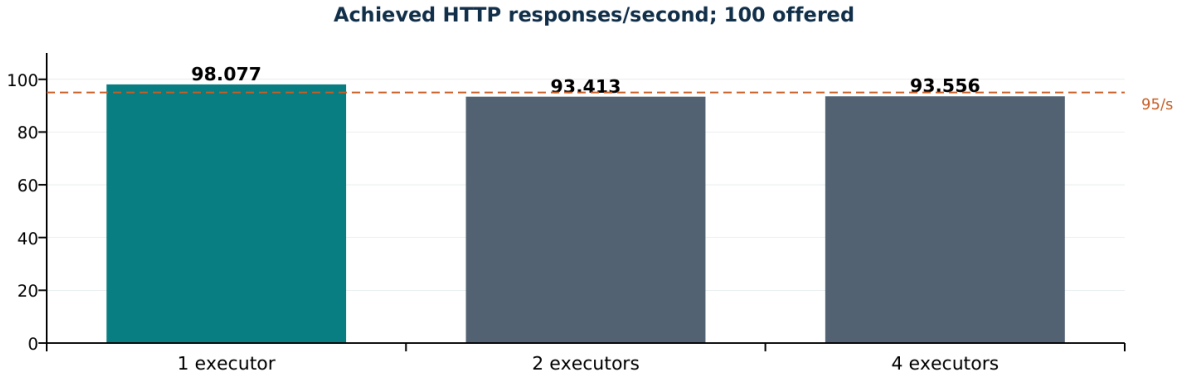
Revision/run	Busy-phase achieved/s	HTTP errors	Interpretation
Baseline	97.300	449	Overload with timeouts
Metadata warming	106.536	148	Fewer errors and metadata reads, still overloaded
Snapshots and lighter monitoring	150.329	0	More work completes, but offers omitted and phase throughput fails
Raw transport/uvloop trial, reverted	131.367	0	Failed offers/throughput; not in final implementation

Metadata reads in the instrumented baseline/warming pair changed from 4,546 to 2,732. Mean sampled host CPU utilization in the snapshot boundary run was approximately 99.38%. The controller experiment measured about 0.953 CPU seconds for three Compose ps calls versus 0.145 for three direct inspections. These observations support targeted reductions of work. They do not provide a complete exclusive CPU profile by component.

The native variable progression retained two failures: before snapshots had three alerts at/above ten seconds and a 10.372-second maximum; with snapshots met alert deadlines but failed some HTTP phases. The later passing run followed the controller improvement. Random temperature paths, rate sequences, cache state, and shared-host conditions differ, so no single-edit speedup percentage is attributed to this sequence.

Executor sizing and polling

More executors did not improve combined capacity on this VM



100 experiments x 10 sensors x 1 Hz, 0.1-second notification poll. Historical local runs, 7 October 2026.

The dashed line is our 95% acceptance criterion. Notification deadlines and data correctness passed in all three.

The October 7 matrix used 100 experiments, ten sensors, one-second samples, 30 carry-out samples, and 100 offered HTTP/s. At a 0.1-second notification poll, one executor achieved 98.077 HTTP/s, two achieved 93.413, and four achieved 93.556. Correctness and notification deadlines passed in those rows; the latter two missed the local HTTP threshold.

One executor was the smallest passing configuration. Among passing tested polling choices, 0.1 seconds had the lowest observed HTTP p95 in that short matrix. It is a measured default, not a proof of an optimal constant. Those runs predate later optimizations, so the matrix explains the decision but is not a current replicated scaling curve.

Tests and operational recovery

The final suite passed **92 tests in 13.12 seconds**, including 14 integration cases. Unit tests cover engine/codec behavior, strict latency boundaries, cache bounds, scheduler fairness, transport, cost calculation, load accounting, and tool-build invalidation. Integration cases exercise real PostgreSQL transaction/version/cache behavior and the public gateway.

The recovery exercise preserved four history points and three notifications through partial readings, replay at new offsets, executor and scheduler restart, two injected notification 503s, Redis loss, and Data/PostgreSQL restart. Clearing Data's RAM cache while Redis was unavailable ensures the final read genuinely exercised persistent recovery.

The live recovery initially found 2,860,538 retained offsets behind the sampled broker positions. Existing progress was preserved; offsets were not reset to latest. Later sampling found zero lag to the sampled broker positions and no pending/failed jobs or quarantined events. The database held 3,028,234 measurements at that final readiness observation. This is evidence of catch-up, not timely delivery of old alerts or recovery of broker-expired input. Readiness.

Capacity runs preceded the final scheduler fairness change. The final tests and real backlog exercise validate that later revision in different ways. A new fixed/variable performance run on its exact source/image identity remains an appropriate future check.

Reproducibility and stronger future experiments

Retain source/image identity, host capacity, background services, cold/warm state, workload parameters, actual event generation, phase-level completions, required/received alerts, maximum latency, queue/lag and disk growth. Preserve failures and reverted experiments. The native test uses random traces; comparing settings is not the same as comparing identical input.

In a scheduled local-test window, repeat key runs at least three times, then change one factor at a time: event concurrency, sensor count, history length, cache state, key skew, or external receiver delay. Hold plateaus long enough to observe backlog growth and recovery. Separate fault-recovery tests from healthy-load latency tests. A longer soak is needed to characterize storage growth, burst credits, and indefinite-duration experiments.

A useful latency budget is sensor completion + broker waiting + scheduler queue + execution/Data commit + outbox waiting + receiver acceptance. Instrument the longest stage before adding workers. Backpressure can preserve memory while that budget still exceeds ten seconds.

Oral exam handbook

Group 15 • Final assessment preparation • 5 November 2026

Practise each answer in your own words, then answer the follow-up without reading. A strong response states a decision, explains the mechanism, points to evidence, and names the relevant limit. Every question can go to any group member.

Requirements and domain behavior

1 What problem does the system solve

It observes laboratory experiments, preserves their temperature history, and notifies required temperature transitions. The room temperature is a complete-sensor mean. Temperature control is another system's responsibility. The scientific value is traceability and timely awareness of excursions; this service does not decide whether an experiment is scientifically valid.

Follow-up: Identify a missing alert, an incorrect mean, and a slow history query as three different failures.

2 Why do we wait for all sensors

The requirement defines temperature as the mean of all allocated sensors for one measurement. Using only the sensors that arrived first changes the quantity. We group by experiment and measurement ID and verify timestamp/hash agreement. Partial readings are durable so replacement workers can finish the same group.

Follow-up: What happens when two of three readings arrive before an executor restarts?

3 Why not use the most recent value from each sensor

Those values can belong to different times. Their mean may never have represented the physical space at any sampling instant. Our grouping preserves measurement identity instead of silently carrying forward old values. A missing sensor delays that group rather than being filled with an invented value.

Follow-up: Explain the effect on both correctness and the ten-second target.

4 What happens during stabilization

Complete groups are monitored and an entry into the inclusive allowed range triggers Stabilized. Going outside and re-entering can trigger another Stabilized. This does not start carry-out automatically; the researcher sends the start event. Stabilization aggregates may be persisted internally but are excluded from required carry-out history responses.

Follow-up: Why distinguish stored internal data from the public query contract?

5 When do we send OutOfRange

Only for the first outside complete mean in a continuous running excursion. Consecutive outside points all belong in outside-history queries but do not each need an alert. Returning in range resets the transition condition, so a subsequent outside point creates a new alert.

Follow-up: How many alerts and outside points result from outside, outside, inside, outside?

6 Are range and time boundaries inclusive

Yes. A temperature exactly equal to either configured threshold is inside. A query includes points exactly at its start or end time, including equal bounds. The snapshot cache uses left and right binary searches to preserve these semantics and duplicate-timestamp ordering.

Follow-up: Why would using the same binary-search side for both bounds be wrong?

7 What can be queried immediately after configuration

Configuration and status are available after the write is acknowledged. An empty history is a valid response before carry-out points exist. Termination does not delete history. A nonexistent experiment and an existing experiment with no points must be distinguished by the API contract.

Follow-up: Find this distinction in the public API/integration tests.

8 What if a sensor or phase event is invalid

The engine rejects unallocated sensors, conflicting duplicates, inconsistent timestamps/hashes, invalid phase order, and transitions with incomplete groups. Kafka input errors are stored in quarantine so the partition can progress. Quarantine preserves evidence; it does not repair the experiment or make its missing data correct.

Follow-up: Why is a rising rejection count a problem even if Kafka lag falls?

9 What is the difference between outside points and intervals

The required outside endpoint returns every outside carry-out measurement. The convenience interval endpoint groups excursions and ends them at the next inside point or termination. An active excursion has no end yet. Neither endpoint's row count is the number of notification delivery attempts.

Follow-up: Can a successful retry change the historical outside points?

10 What do we do with the measurement hash

We check that sensors in a group agree and forward the original opaque value as notification `cipher_data`. We do not decrypt it or derive temperatures from it. The original notification validator uses its own keys and producer truth to check correctness; our independent sink is a different test double.

Follow-up: Why is passing the original validator stronger contract evidence than only using our sink?

Sources: engine, models, engine tests, assignment.

Architecture and scaling decisions

11 Why separate these services

They have different obligations: intake preserves order, executors calculate, Data commits and serves history, the gateway exposes routes, and the notifier tolerates receiver delays. Separation allows targeted replacement/scaling. Its cost is more network calls and operations; on one VM a modular monolith could be simpler and cheaper to execute.

Follow-up: Which component should scale if only external notification delivery becomes slow?

12 Why Python

The work includes substantial network/database waiting, so asynchronous I/O is useful. Python also makes the state machine and validation easy to inspect. Suitability is established by the measured workload, not by claiming the language is universally fast. CPU-bound processing and shared-host contention still require measurement.

Follow-up: Why would changing language not automatically fix an overloaded database?

13 What is stateless about an executor

It has temporary load counters and connections but owns no authoritative long-lived experiment state. Partial groups, prior range status, versions, and completed identities are loaded from Data. A replacement can recompute a task. The tradeoff is extra persistence/network work compared with keeping experiments permanently in worker RAM.

Follow-up: Where is a two-of-three-sensors partial group after a successful task?

14 Why PostgreSQL and Redis together

PostgreSQL supplies authoritative transactions, uniqueness, indexes, and concurrency control. Redis caches disposable serialized query results. Redis-only authority would need another durability and atomicity strategy. Redis loss increases database work but does not erase the experiment history; recovery testing exercises that fallback.

Follow-up: Why did the recovery test also restart Data after adding the RAM cache?

15 Why keep JSONB state and relational measurements

JSONB conveniently represents evolving state, including partial sensor groups. Relational measurement columns and indexes support ordered time queries and identity uniqueness. This is a deliberate hybrid. Putting everything into an unindexed JSON document would make query and constraint behavior harder to control.

Follow-up: Name one uniqueness constraint and one history index.

16 Why Caddy when api.py exists

The deployed gateway forwards public queries directly to Data and configuration to an executor, reducing Python proxy overhead. The Python API module still defines the development interface/OpenAPI. Internal commit and notification-claim routes remain off the public gateway. Both routes and error behavior need contract tests.

Follow-up: Which service actually reads PostgreSQL for a public history query?

17 Why least-busy scheduling

It is simple and appropriate for broadly comparable batches. Reported active work plus local reservations avoid assigning every task to a worker whose reported load is stale. Worker capacity remains enforced. Batches can differ in sensors and database work, so this is not an exact prediction of completion time.

Follow-up: What protects the executor if the scheduler's load observation is old?

18 Why did we deploy one executor

The earlier same-host matrix found one executor achieved 98.077 HTTP/s against 100 offered, while two/four achieved about 93.4–93.6. Additional workers competed for shared resources. One was the smallest passing configuration. That historical result explains the choice but should be remeasured on new hardware or changed workloads.

Follow-up: Distinguish a scalable worker abstraction from a measured capacity gain.

19 What does the autoscaler actually do

It is a host-side opt-in policy, not continuously enabled in the demo. Defaults are one to four replicas, scale-up after five seconds of queue above 1,024 or occupancy at least 75%, and scale-down after sixty seconds fully idle. It cannot distinguish every downstream bottleneck.

Follow-up: Why might scaling up during CPU saturation reduce HTTP throughput?

20 What limits concurrent tasks

One active task per Kafka partition preserves order. With P partitions, K executors and capacity C, the bound is min(P, K×C). At 16 assigned partitions and one capacity-eight executor, it is eight. This is a concurrency bound; batch service time and skew determine throughput.

Follow-up: Why can one hot partition still limit progress with many idle workers?

Sources: scheduler, discovery, autoscaler, gateway, replica matrix.

Consistency and failures

21 What is the most important transaction

The Data transaction commits state versions, measurements, notification obligations, rejected input, and the partition checkpoint together. It prevents durable progress from claiming that work is done while history or an alert obligation is missing. Kafka acknowledgement follows this commit.

Follow-up: What would break if the outbox job were created only in notifier RAM?

22 Why store a database checkpoint as well as Kafka offsets

Kafka and PostgreSQL do not commit atomically together. The database checkpoint identifies effects already durable if Kafka acknowledgement is lost. Replay can safely skip them and later advance Kafka. Logical measurement identities also prevent duplicate aggregates when completed measurements reappear at new offsets.

Follow-up: Which duplicate case is not solved by comparing only Kafka offsets?

23 What if we acknowledge Kafka first

A crash before the database commit could leave Kafka pointing past events whose history and notification jobs do not exist. Ordinary consumption would skip those effects. Our order is durable application commit, then Kafka next-read offset. The remaining replay window is handled by deduplication.

Follow-up: What offset does Kafka store after durably processing offset 45?

24 What prevents concurrent writers from losing updates

Commit proposals carry expected state versions and checkpoint. Data checks them inside the transaction. A conflicting version aborts the whole proposal, and the executor reloads/recomputes. Deterministic state-write ordering also reduces conflicting lock order. It is not acceptable to merge arbitrary partial results without checking their assumptions.

Follow-up: Describe the stale-middle-of-three integration regression.

25 Why was batching safe

It reduced sequential state-write round trips while preserving sorted lock order and checking every returned version. A mismatch still rolls back all effects. The regression test proves all-or-nothing behavior when one middle update is stale, then verifies successful progress after the version is corrected.

Follow-up: Why is checking only the first returned update insufficient?

26 What happens to duplicate sensor messages

An identical repeated reading within a pending group occupies the same sensor entry; it is not counted twice. A conflicting value is rejected. Completed measurement identities suppress re-emission of completed aggregates/jobs, including logical replay at new offsets. These are distinct duplicate cases with distinct checks.

Follow-up: Where are completed identities retrieved for a task?

27 What if an earlier sensor never arrives

Its incomplete group prevents later complete groups for that experiment from draining in order. Pending-group limits prevent unbounded accumulation, and invalid transitions are rejected. This protects correctness but can prevent timeliness. The product needs an explicit missing-sensor policy before choosing interpolation, timeout completion, or partial means.

Follow-up: Why would silently averaging available sensors change the requirement?

28 Are notifications exactly once

Database jobs are unique and durable. External acceptance has a separate ambiguity: the receiver can accept before the sender loses its response or crashes. Retrying can duplicate acceptance. We send an idempotency key, but exactly-once external effect requires the receiver to honor it.

Follow-up: Why does a PostgreSQL transaction not eliminate that remote crash window?

29 What do notification leases guarantee

They temporarily assign a pending job and allow retry after expiry. Result updates require the current lease token, preventing a stale owner from acknowledging another owner's work. The fifteen-second lease supports recovery but may exceed the ten-second latency target after a crash; recovery correctness is not an outage SLA guarantee.

Follow-up: Why is an arbitrarily tiny lease also risky?

30 What happens when Redis or PostgreSQL fails

Redis loss falls back to authoritative SQL after bounded cache failure handling. PostgreSQL loss prevents new durable commits, so the scheduler cannot safely acknowledge those events; backlog/retries grow. The tests establish specific restart recovery cases. They do not provide cross-host redundancy or a guarantee through indefinite outages.

Follow-up: Which symptoms distinguish cache failure from loss of the authority?

Sources: Data, executor, notifier, schema, recovery evidence.

Performance and experimental reasoning

31 What is our strongest current performance result

The original tools' native variable run used 100 main experiments, ten sensors, and one-second samples. All five observed HTTP target phases passed, all 703 expected alerts arrived, maximum delay was 2.884 seconds, and 202 complete history comparisons passed. It is a specified local result, not the hosted professor's grade.

Follow-up: Which exact target rates were observed, and did that sequence include 200?

32 Can we sustain 200 requests per second

The native fixed target passed our local rule at 190.573 achieved/s, with a maximum alert delay of 8.958 seconds. The independent live-history 200 peak failed at 150.329/s. Different pacing, query timing, history state, and receiver paths matter. The evidence does not support an unrestricted 200-rps claim.

Follow-up: What is the native fixed run's margin above our local throughput threshold?

33 Why do notification counts differ between native runs

The original simulator produces random temperature paths. The number of range transitions, and therefore required alerts, differs even with the same experiment/sensor/rate settings. Each run must compare received with its own expected set. Equal alert counts are not a prerequisite for comparable configured workloads.

Follow-up: Why is comparing only received counts weaker than expected-versus-received validation?

34 What does 202 history checks mean

Two full history endpoints were checked for 101 completed experiments: 100 main experiments plus one warmup completed before the main concurrency. It is not 202 concurrent experiments, and it is separate from the thousands of continuous HTTP validator responses.

Follow-up: Why does completing warmup first matter for the 100-concurrent-experiment claim?

35 Why is p99 below ten seconds insufficient

A strict deadline can be violated by a small tail that a percentile hides. Missing alerts may not appear in a latency histogram at all. We need required/received counts, validation correctness, and every accepted alert below ten seconds. Exactly ten fails the new predicate and deadline-miss counter.

Follow-up: How would a dashboard look if no alerts were received at all?

36 What does keeping up with HTTP mean in our tests

We compare valid completions with offered rate, track errors and omitted requests, and include drain time. The local threshold is 95% of target; the rubric does not prescribe that percentage. Queue growth and tail latency still matter, especially when a short run only narrowly meets the local rule.

Follow-up: Why cannot a target-rate gauge alone prove throughput?

37 Why use an open-loop test as well as the original generator

Independently scheduled requests continue to represent offered demand when responses slow. A closed-loop client can reduce its own effective offered load while waiting. Our open-loop result records intended/scheduled/completed counts, planned-time latency, and phase drain; a bounded pending queue fails explicitly if it omits work.

Follow-up: Explain coordinated omission without relying on the term itself.

38 How can zero HTTP errors still be a failure

The independent 200 peak intended 6,000 requests but scheduled 5,526. They all succeeded, yet 474 offers were omitted and completions drained beyond thirty seconds. Achieved rate was 150.329/s, below 190. Correct responses are necessary, but completing fewer requests late does not establish capacity.

Follow-up: Calculate 5,526 divided by approximately 36.759 seconds.

39 Did we statistically prove the optimizations caused the gain

No. We have before/after observations, mechanism-level reasoning, regression tests, and diagnostic measurements. Native random traces and shared-host conditions differ, and changes were not all isolated. Repeated controlled runs with matched inputs would strengthen causal attribution. Failed and reverted trials remain in the evidence.

Follow-up: Name one confounder and one practical way to control it.

40 Which workloads remain unproven

Arbitrary long histories, prolonged load/credit behavior, all variable event-arrival schedules, every partition skew, and the professor's external workload are not established by short local passes. Capacity runs also preceded the final scheduler fairness revision. Repeat representative tests on that exact revision in a separate test window.

Follow-up: Why is a longer soak relevant even if memory caching is bounded?

Sources: native variable, native fixed, independent failure, phase accounting, validation.

Optimizations and bottleneck diagnosis

41 What does the new history cache save

For small histories it retains a versioned ordered snapshot, allowing many different time windows to avoid repeated SQL/Redis work. Binary search selects inclusive bounds. Histories above 512 points fall back to indexed SQL/Redis. The cache is bounded by 16,384 points and 256 entries, so arbitrary experiment duration cannot fill RAM indefinitely through this cache.

Follow-up: Why probe 513 points instead of returning the first 512?

42 Why do metadata and history come from one SQL statement

Separate statements could pair an older version with newer rows or vice versa under concurrent writes. One statement gives a consistent database snapshot for both. The cache then associates its content with the correct version. This is a correctness condition, not merely an optimization detail.

Follow-up: What happens when a request needs a version newer than a cached snapshot?

43 Why is commit-time metadata warming fenced

Writer A can commit, release its connection slowly, and resume after writer B has already committed. Without a generation check, A could publish older metadata last. Active-writer and generation fences allow only the final uncontended writer to warm known committed values. A forced-interleaving integration test protects this case.

Follow-up: Why is checking only whether A's SQL commit succeeded insufficient?

44 Why not horizontally scale Data immediately

Its local metadata/snapshot coherence assumes one process owns all writes. Separate processes can retain different versions without shared invalidation. Adding Uvicorn workers is also multiple processes. We would first introduce a shared version/invalidation protocol, remove local metadata caching, or redesign reads, then test concurrency and failure behavior.

Follow-up: Why does having Redis somewhere in the architecture not automatically coordinate RAM caches?

45 What did monitoring itself do to performance

The recovered outbox was large, and repeated historical counts scanned substantial data. We indexed status and slowed historical sent/rejected refresh to once per minute while retaining faster pending/failed/age checks. Monitoring should detect deadline problems without competing unnecessarily with the work it observes.

Follow-up: Which dashboard totals can now be slightly stale, and why is that acceptable?

46 How do pool wait and occupancy differ

Pool wait measures time to acquire a connection. Occupancy measures time while holding it, including transaction/SQL work, application work, and scheduling. High wait may indicate a busy pool, but enlarging it can increase database contention. Neither metric alone is a pure SQL execution profile.

Follow-up: What extra evidence distinguishes slow SQL from event-loop scheduling pressure?

47 Why optimize the test controller

It shared the two CPUs with the service. Repeated Compose inspection was much more expensive than direct container inspection. Caching IDs and inspecting both load containers together reduced that overhead while keeping workload and validation. A test driver that consumes capacity can distort the service's measured boundary.

Follow-up: Why should a dedicated load-generator host still record its own saturation?

48 Why did we revert the raw transport and uvloop trial

It did not improve the independent boundary workload: its busy phase achieved about 131.367/s versus 150.329 in the snapshot run, and still omitted offered work. These are unreplicated observations, but there was no evidence to retain extra complexity. The failed result remains archived and is not described as final code.

Follow-up: What experiment would justify reconsidering that change?

49 What problem did per-partition backpressure solve

A hot partition could consume the global queue budget and pause intake for other partitions despite spare worker capacity. Per-partition high/low watermarks let other partitions continue, with the global guard retained. The queue count includes in-flight work. It improves fairness; it does not parallelize the hot partition itself.

Follow-up: At 16 partitions and default settings, calculate the per-partition high watermark.

50 How would you identify two different bottlenecks

If CPU, HTTP delay, and scheduler backlog rise together, inspect shared process/generator CPU and compare at fixed workload. If histories are current but outbox age and 503s rise, inspect the receiver/network. The first can benefit from reducing compute/coordination work; the second needs delivery-path action, not more executors.

Follow-up: State what observation would falsify each initial hypothesis.

Sources: Data, history, cache/race tests, controller measurements, reverted trial, scheduler tests.

Operations, costs, and assessment

51 What was wrong with course Kafka connectivity

The provider's broker certificate names did not match the public DNS name used by the demo. The demo overlay disables hostname matching while retaining CA trust and client-certificate authentication. This is a specific compatibility exception and weakens hostname identity checking; valid public-name certificates should restore strict matching.

Follow-up: Distinguish TCP reachability, TLS identity, client authentication, and partition assignment.

52 What topic and group do we use for the professor

The demo uses `experiment` and `group15-temperature-demo-v1`, with the external course notification endpoint. The original-tool self-test uses a local topic/group/receiver. The modes share the application database, so stopping generators alone does not switch the application back to demo configuration.

Follow-up: Why should pending outbox jobs be inspected before switching receivers?

53 Which dashboard parts are ours

The operational dashboard on loopback 3004 is ours. The local course dashboard on 3009 uses the instructor panels and local datasources. We added the bottom panel named Latest local test, driven by our explicit checks. A pinned old local success is not the hosted professor's current result.

Follow-up: What would you check before interpreting a green panel as evidence?

54 What did the Docker cleanup remove

Killing stopped processes; removing containers removed their writable layers; removing images removed executable layers/references. Named volumes survived the listed commands. PostgreSQL later reopened successfully, so preservation is now demonstrated more strongly than merely listing volume names. Disk/VM loss still needs a real off-host backup strategy.

Follow-up: Why is pruning volumes dangerous after all containers have been removed?

55 What did backlog recovery establish

The existing group had 2,860,538 retained offsets behind the initial sampled broker positions. We processed retained work without resetting to latest and eventually reached sampled positions with zero sampled lag. Old alerts remained late. Broker-expired events are outside what this catch-up can reconstruct.

Follow-up: Why is the scheduler's queue of zero not the same as a complete broker lag measurement?

56 How much does the deployment cost

The recorded estimate is one VM plus provisioned disk: hourly VM price plus disk quantity times monthly price divided by 730. With the recorded rates and unconfirmed 20 GiB gp3 assumption it is about $0.043211/hour. Credits, IPv4, traffic, tax, discounts, and course infrastructure are excluded; it is not the bill.

Follow-up: Why should four containers on this VM not be billed as four VMs?

57 Why does disk still need a plan

At 100 experiments and 1 Hz, uninterrupted carry-out creates 8.64 million aggregate rows per day, before indexes, jobs, and other storage. Bounded cache memory does not bound persistent history. Retention limits cover some disposable infrastructure, but required history needs capacity, export/archival policy, and tested backups.

Follow-up: Why is a same-disk backup insufficient for disk loss?

58 What do the 92 tests prove

They cover specific correctness, API, cache-race, scheduler, validation, and recovery-related behaviors; fourteen use integration infrastructure. They do not prove all inputs, indefinite runtime, all failures, or hosted workload performance. The load/recovery artifacts provide separate kinds of evidence. This report preparation did not rerun workloads or change application code.

Follow-up: Choose one regression test and state the failure it would catch.

59 How do we aim for Excellent

Show a healthy working demo, correct data and strict alerts under the applied workload, and HTTP capacity with honest phase accounting. Explain distinct responsibilities and identify multiple bottlenecks using evidence. Train every member across the whole system. More report pages or containers cannot replace working behavior and individual understanding.

Follow-up: Which remaining experiment most directly strengthens our capacity claim?

60 What did you personally contribute

Answer with your real task, the decision you made, a file/result/review that substantiates it, and how you verified it. Code, experiments, operations, analysis, and documentation can all be meaningful contributions. Suggested study roles are not authorship evidence. Follow the course's applicable disclosure rules for tools and assistance.

Follow-up: Explain one component outside your preparation specialty without handing off the answer.

Sources: demo configuration, readiness, cost evidence, rubric, team rehearsal.

Worked exercise A Calculate the observable behavior

Use three sensors, range 25–26 inclusive, and complete matching groups. Assign running samples M1–M5 timestamps 10–14 respectively, followed by termination at 15.

Phase and sample	Sensor values	Mean	Notification
Stabilizing S1	20, 21, 22	21	None
Stabilizing S2	24, 25, 26	25	Stabilized
Running M1	24, 26, 28	26	None
Running M2	26, 27, 28	27	OutOfRange
Running M3	27, 28, 29	28	None

Phase and sample	Sensor values	Mean	Notification
Running M4	24, 25, 26	25	None
Running M5	22, 23, 24	23	OutOfRange

Expected outputs: five carry-out history points; three outside points M2, M3, M5; two carry-out notifications and one Stabilized. Query bounds [11,12] return M2 and M3. Excursion intervals are 11 to 13 and 14 to 15; the end identifies return-in-range or termination, not an assertion that the closing point is outside.

Remove sensor C from M2. No M2 mean is available; M3 waits behind it even if complete. Duplicate A with the same value adds no second contribution. Duplicate A with a conflicting value is rejected. Each member should derive all four variations independently.

Worked exercise B Trace the two crash boundaries

Start with application checkpoint 40. A task processes 41–45, completes a measurement, and creates a job. Data commits 45; the response is lost. Replay finds durable progress and does not create a second measurement/job. Kafka can acknowledge next-read offset 46. If the transaction had rolled back, checkpoint 40 would instead permit recomputation.

Now move the crash to after the external receiver accepts but before local result acknowledgement. Database input progress cannot resolve that remote outcome. Retry may repeat acceptance. State why idempotency belongs at the receiver and why the lease token only protects local job ownership.

Worked exercise C Interpret load honestly

A 200/s phase lasts thirty seconds. It expects 6,000 offers, schedules 5,526, receives 5,526 valid responses, and drains at 36.759 seconds. Achieved rate is approximately 150.329/s. It fails both complete offering and the local 190/s rate threshold despite zero response errors.

A native 200/s run instead achieves 190.573/s with zero validator errors. It passes that local threshold narrowly. Explain why these different workloads cannot be combined into a claim that the final architecture sustains every 200/s request pattern.

Worked exercise D Diagnose before changing capacity

Observation	Hypothesis to test	Evidence that would redirect you
CPU high, intake/HTTP backlog rises	Shared compute/driver contention	CPU mostly idle but database pool waits dominate
Histories current, old pending jobs, receiver 503s	External delivery bottleneck	Receiver fast but Data claim/result requests are slow
Free executor slots, one lagging partition	Key skew or sequential hot partition	Balanced partition lag with all Data operations slow
SQL recovery appears successful with Redis off	Persistent store served the query	The response was supplied by an un-cleared Data snapshot cache

For each row, name a metric, a source function, and one targeted experiment. Do not diagnose from one graph without a competing explanation.

Team rehearsal and presentation plan

Group 15 • Final assessment preparation • 5 November 2026

The group combines Computing Science, Data Science, and Business Informatics. Use these backgrounds as teaching strengths, then rotate explanations until each member can defend the whole project. The rubric explicitly permits redirected questions and different individual grades. This plan assigns preparation responsibilities, not historical authorship.

Shared readiness standard

Every member should independently draw the architecture, calculate a complete-sensor example, explain database/Kafka replay and the external duplicate window, read a raw benchmark, distinguish two bottlenecks, and locate the relevant implementation/test. Memorizing component names is insufficient if the member cannot predict behavior after an input or failure changes.

Preparation lead	Teach the others	Learn back from the others
Computing Science	Ordering, atomic commit, versions, cache races, backpressure	Statistical limits, workload accounting, cost and business risk
Data Science	Offered versus achieved rate, maxima/percentiles, random traces, confounders, capacity experiments	Checkpoint/outbox and cache coherence from actual code
Business Informatics	Requirement traceability, value, tradeoffs, cost scope, incident decisions	Complete grouping, concurrency, and performance arithmetic

Scientific understanding here includes experiment design and evidence quality; the assignment does not require machine learning. Business analysis includes precise service contracts and operational tradeoffs; it is not restricted to presenting the introduction.

Four rehearsal sessions

Session 1, 45 minutes: derive the behavior. Read the submission report and draw the diagram independently. Each member calculates exercise A in the handbook, then introduces a missing sensor, duplicate, and boundary value. Finish by explaining why three outside points can require only two alerts.

Session 2, 60 minutes: defend the implementation. Open `engine.py`, `executor.py`, `data.py`, `scheduler.py`, and `notification.py`. Rotate the person explaining each function. Place a crash before commit, after commit, and after remote acceptance. Then trace the delayed-writer metadata race and the stale-middle-version test. Every explanation must identify an invariant and a test.

Session 3, 45 minutes: challenge the evidence. One person states a claim, another finds its JSON values, and the third challenges the inference. Rotate using 703/703, 202 comparisons, 190.573/s, 5,526/6,000 offers, and the independent failed recovery phase. Ask why the native variable run's configured maximum is not an observed 200 phase and why final scheduler changes need separate evidence.

Session 4, 40 minutes: mock assessment. Deliver the ten-minute practice presentation below, then spend twenty minutes on random redirection and ten minutes correcting weak answers. One member answers, one acts as examiner, and one checks mechanism/evidence; rotate after every question. Ten minutes is a rehearsal format, not a confirmed exam allocation.

If only one hour remains: ten minutes quick reference, twenty minutes architecture/crash examples, twenty minutes random questions, and ten minutes demo/evidence navigation. This is a fallback study plan, not equivalent to the full rehearsal.

Ten-minute presentation

Time	Rehearsal lead	Message and artifact
0:00–1:00	Business Informatics	Monitoring-only problem, complete-sensor mean, traceability and transition alerts
1:00–2:30	Computing Science	Architecture diagram, responsibilities, credible monolith alternative
2:30–4:00	Computing Science	One transaction and two crash boundaries; correctness before optimization
4:00–6:00	Data Science	Native variable/fixed results and independent failed boundary; exact workload and counts
6:00–7:30	Data Science	Database/read/controller bottlenecks and targeted revisions; limits of causal claims
7:30–9:00	Business Informatics	Disk incident, live recovery, cost assumption, remaining failure domain
9:00–10:00	Any member	Show current readiness and state the next discriminating experiment

Use the architecture, failure-boundary, HTTP-phase, and replica figures supplied with the pack. Keep one raw result and one source function ready to open. The complete code appendix is a reference for questions, not something to scroll through during the main explanation.

Suggested opening: “We preserve complete-sensor experiment history and notify the required temperature transitions. We separated ordered intake, computation, durable storage, queries, and delivery. The core guarantee is that state, history, alert obligations, and input progress commit together. We measured the system, found avoidable database and coordination work, and improved it while retaining the consistency checks.”

Suggested result statement: “Our native variable run checked five observed HTTP targets with 100 experiments, ten sensors, and one-second samples. All 703 required alerts arrived, maximum 2.884 seconds, and both full histories matched for 101 experiments. The separate 200 target passed narrowly, while an independent live-history 200 peak still failed. We retain both results because workload definition matters.”

Suggested limitation statement: “Responsibilities can scale separately, but the deployment shares one VM and Data's invalidation currently assumes one writer process. A hot partition, a saturated database, and a slow notification receiver need different remedies. Our next performance evidence should repeat representative runs on the exact final scheduler revision and then vary the identified limiting factor.”

Redirection and challenge cards

- Ask Business Informatics to explain the delayed-writer cache race and what the generation check prevents.
- Ask Data Science to trace durable offset 45 versus Kafka next-read offset 46.
- Ask Computing Science why before/after maxima do not isolate causal improvement.
- Ask Business Informatics why 5,526 error-free responses can fail a 6,000-offer phase.
- Ask Data Science why clearing Data's cache matters in a PostgreSQL recovery test.
- Ask Computing Science to distinguish provisioned storage cost from database-used bytes.
- Ask any member whether a zero scheduler queue proves zero broker lag.
- Ask any member why the external hostname exception is a compromise.
- Ask any member whether reducing a fifteen-second lease guarantees the notification SLA.

- Ask any member which test actually measures the final scheduler revision's capacity.

For each answer the observer records the incorrect assumption, source/evidence that resolves it, and a new example to retry five minutes later. Reward a precise correction, not confidence without support.

Score understanding without inventing grades

Use 0 = cannot explain, 1 = terms without application, 2 = correct explanation and example, 3 = mechanism plus evidence, tradeoff, and counterexample. These are study scores, not a mapping to the professor's marks. Aim for every member at least 2 on every row.

Topic	Computing Science	Data Science	Business Informatics
Domain and interfaces	Unassessed	Unassessed	Unassessed
Partition order, versions, replay	Unassessed	Unassessed	Unassessed
Outbox and external duplicate window	Unassessed	Unassessed	Unassessed
Snapshot and metadata coherence	Unassessed	Unassessed	Unassessed
Workloads, arithmetic, experimental limits	Unassessed	Unassessed	Unassessed
Bottleneck diagnosis and scaling	Unassessed	Unassessed	Unassessed
Demo modes, costs, and recovery	Unassessed	Unassessed	Unassessed

Actual contribution record

Complete this from real work before using it in a submission. Include implementation, experiment execution, analysis, reviews, operations, documentation, or teaching that actually occurred. Link commits/files/results or dated review notes. Do not assign authorship from the preparation leads above.

Member	Actual work and decision	Artifact and verification
Computing Science member	To be completed by the member	To be linked to evidence
Data Science member	To be completed by the member	To be linked to evidence
Business Informatics member	To be completed by the member	To be linked to evidence

When uncertain in the exam, state the assumption, reason from state ownership and the failure boundary, then identify what you would inspect. A specific evidence limit is a stronger answer than inventing an observation. Follow the course's applicable assistance/disclosure policy when describing how the work was produced.

Demo readiness and panel runbook

Group 15 • Final assessment preparation • 5 November 2026

The fresh report-preparation check found healthy API, scheduler, and both Grafana instances, 16 assigned partitions, zero application queue, and stopped local generators. Demo input was `experiment`, consumer group `group15-temperature-demo-v1`, and delivery used the external course endpoint. This dated record does not replace a check immediately before the exam.

Read-only checks before the professor starts

From the repository root:

```
curl --fail http://127.0.0.1:3003/health
curl --fail http://127.0.0.1:8001/health
curl --fail http://127.0.0.1:8001/status
sudo -n docker ps --format '{{.Names}} {{.Status}}'
df -h .
```

Inspect actual topic/group, receiver, assigned partitions, queue trend, and notification failures. The earlier broker watermark check found zero sampled lag; the status endpoint's application queue alone cannot prove that all retained broker work is consumed. A separate preflight can test course authentication with its own temporary consumer group:

```
.venv/bin/python scripts/check-course-kafka.py --topic experiment
```

That script neither publishes messages nor commits the live group's offsets. It confirms a temporary consumer's connection/assignment, not that the actual scheduler is processing correctly.

Read outbox state without printing payloads or credentials:

```
sudo -n docker compose --env-file .env.monitoring \
  -f compose.yml -f compose.demo.yml exec -T postgres \
  psql -U observability -d observability -c \
  "SELECT status,count(*) FROM notification_outbox GROUP BY status;"
```

A group member should also query the public API from their laptop using the VM address and a known real experiment ID. Localhost health does not establish the professor's firewall/routing path. Choose a real history time window and compare meaningful values; an empty array for an unobserved period is not a consistency demonstration.

Open the two local panels

Run this on your laptop with your existing SSH key/options:

```
ssh -N -L 3004:127.0.0.1:3004 -L 3009:127.0.0.1:3009 \
  ubuntu@YOUR_VM_PUBLIC_IP
```

Open the operational dashboard at `http://localhost:3004` and local instructor-style dashboard at `http://localhost:3009/d/cec-course-grading`. Use the existing Grafana credential setup described in MONITORING.md; do not paste private credentials into screenshots or reports.

View	What it demonstrates
Operational Grafana, 3004	Queue, delivery, cache, pool, CPU/disk, and cost observations
Local instructor-style Grafana, 3009	Original panels over local test truth and metrics; completed-run time range
Latest local test, bottom panel	Our extra explicit pass/fail checks for the selected saved run
Course-hosted Grafana	The course environment's own current observations; not implied by a local pass

The runner pins local Grafana to a completed test window. Inspect the time range and nonzero workload before interpreting green. `course_test_passed=1` is our local metric, not a professor-issued grade. Saved

benchmark JSON/PDFs remain evidence after live metrics age out under retention.

Keep demo and test modes distinct

Mode	Topic and destination
Professor demo	External `experiment`, existing demo consumer group, external notification API
Independent development	Local `experiment`, local notification sink
Original-tool self-test	Local course-test topic/group, original Rust notification validator

These modes share the application project/database. `make course-test`, `make course-stress`, and `make course-variable` change local deployment configuration and create workload; run them only in an intentional local-test window. `make course-stop` stops generators while preserving panels/results. It does not restore demo topic or receiver by itself.

If tests were running, stop them with:

```
make course-stop
```

Before changing receiver mode, check that pending jobs are understood and drained to the correct receiver. Preserve and investigate unexpected jobs; blindly deleting them or sending local validator jobs to the external endpoint breaks the evidence and contract.

Restore demo configuration when needed

This is an operational recovery action, not a step to execute against an already healthy professor test. Reuse existing database credentials and volumes. With the prepared `.env.monitoring` and course credential files present:

```
sudo -n docker compose --env-file .env.monitoring \
  -f compose.yml -f compose.demo.yml -f compose.monitoring.yml \
  up -d --wait postgres redis data executor scheduler notification api \
  prometheus node-exporter grafana
```

If images are absent, rebuild the application first using the documented deployment instructions. Do not change the consumer group merely to discard old input, and do not reset offsets to latest to hide lag. Inspect both broker retention and event age when recovering a long outage.

Local `kafka`, `notification-sink`, and `course-notifications` are test infrastructure; they can be stopped after a completed test window when returning to external demo mode. Preserve their named volumes and keep the local panel/database if you want to display archived results. Do not issue an indiscriminate command that kills all containers.

Disk incident and recovery policy

The user's cleanup removed containers/images but retained named volumes. Recovery reopened PostgreSQL with more than 2.3 million measurements and later processed retained Kafka backlog. The earlier outage's missed deadlines remain missed. Complete reconstruction of input expired from Kafka is not established.

Logs and local Kafka/Prometheus retention now have limits; disk alerts warn below 15% and become critical below 8%, with a separate projected-exhaustion alert. Active segments/WAL and persistent history mean retention settings are not exact total-disk caps. Infrastructure alerts are visible in Prometheus; external alert delivery has not been configured.

Inspect capacity before compiling tools or adding optional workloads. Preserve authoritative data; never free space by deleting files inside PostgreSQL's volume. A tested off-host backup/restore program remains future work. Disposable build cache can be removed after preserving the compiled instructor-tool image; source/image fingerprint reuse avoids a normal no-change test recompiling Rust. `--rebuild-tools`

intentionally forces a rebuild and may need substantial space.

A concise live demonstration

Show fresh health and actual demo mode, then a meaningful history query, current workload/alert evidence, and the bottleneck dashboard. Explain a failing historical capacity test as part of the measured design process. Be ready to open the exact JSON and the relevant code/test. All members should know how to stop local generators without stopping the actual service.

A current healthy service plus an old local green panel establishes two different facts. Only the professor's current evaluation establishes the result of that applied workload.

Exam quick reference

Group 15 • Final assessment preparation • 5 November 2026

Rubric: demo 70%, architecture 30%. Excellent adds variable HTTP capacity and identifying different bottlenecks under load. A working demo is required. Any member can receive any question.

System: Kafka -> ordered scheduler -> stateless executor -> Data -> PostgreSQL. Caddy exposes public queries; Redis and bounded snapshots accelerate reads; a separate notifier drains the durable outbox. The system observes, not controls, temperature.

Correctness to explain in one minute

Use all allocated sensors for the same measurement identity, timestamp, and hash. Inclusive range boundaries are in range. Stabilization entry/re-entry can notify; carry-out starts on the researcher's event. Required history includes carry-out only. Three consecutive outside points produce three outside-history points but one excursion alert.

State, measurements, jobs, rejected input, and checkpoint commit together before Kafka acknowledgement. After processing offset 45, Kafka's next-read offset is 46. Lost acknowledgement allows deduplicated replay. External acceptance followed by a lost response is a separate duplicate window requiring receiver idempotency.

Values to remember

Fact	Value
Demo	`experiment`; `group15-temperature-demo-v1`; external course notifications
API and panels	API 3003; operational Grafana 3004; local course Grafana 3009
Execution	One executor, capacity eight; 16 assigned partitions at latest check
Buffering	Batch 256; global high/low 8192/4096; per-partition share 512 at 16 partitions
Delivery	Empty poll 0.1 s; concurrency 32; timeout 2 s; lease 15 s
Snapshot cache	At most 512 points per small history; 16,384 total points; 256 entries
Final suite	92 passed, including 14 integration cases

Results with scope

Test	What to say
Native variable	100 experiments × 10 sensors × 1 Hz; targets 148/182/181/114/106 passed; 703/703 alerts, max 2.884 s; 202 history checks
Native fixed 200	190.573 valid responses/s; 596/596 alerts, max 8.958 s; narrow local pass
Independent 25/100/50	All phases passed; peak achieved 97.926/s; 1,626/1,626 alerts, max 6.189 s
Independent 50/200/100	Failed: peak achieved 150.329/s; 474 of 6,000 peak offers omitted despite zero HTTP errors

Our HTTP threshold is 95%, **not a percentage specified by the rubric**. All alerts must be below ten seconds; p99 alone and zero missing data alone are insufficient. Native random traces differ. Capacity runs preceded the final scheduler fairness change, which has separate test/backlog evidence.

The improvement story

Sequential state-write round trips caused overhead; pipelining retained all version checks. Larger recovered history exposed repeated metadata/history reads and expensive monitoring counts; versioned snapshots, writer fencing, and indexed/slower historical counts reduced work. Repeated Compose inspection competed for CPU; direct cached inspection reduced controller overhead. A raw transport/uvloop trial did not help and was reverted. Hot-partition buffering during catch-up motivated partition watermarks.

Earlier replica tests achieved 98.077/s with one executor versus about 93.4–93.6 with two/four on the same host. More containers did not create more CPUs.

Limits and operational facts

One Data process owns local cache invalidation; extra Data/Uvicorn workers need a shared coherence design. One VM/disk is a failure domain. A 15-second lease can miss a ten-second deadline during recovery. Indefinite history requires storage/backup planning. Infrastructure alert rules exist, but external alert delivery is not configured.

The deployment was recovered and the existing group caught up to sampled broker positions. Old outage alerts stayed late; broker-expired input cannot be reconstructed by catch-up. Fresh report checks found healthy API/panels, 16 assignments, empty application queue, and stopped local generators. The **Latest local test** bottom panel is ours and is not the professor's grade.

Cost estimate: one VM plus provisioned disk/730 hours, about $0.043211/hour under the recorded unconfirmed gp3 assumption; excludes credits, IPv4, traffic, tax, discounts, and course services.

Diagnose before scaling

CPU plus HTTP/intake queues -> shared resources. Pool wait plus long connection occupancy -> investigate Data/database. Current history plus old outbox/503s -> delivery path. Free workers plus one lagging partition -> skew/sequential work. Disk growth -> capacity/preservation, not extra executors.

Input rate E×S/T; aggregate rate E/T; concurrency bound min(P,K×C). At 100 experiments and 1 Hz, continuous carry-out creates 8.64 million aggregate rows/day.

Answer structure: decision, mechanism, evidence, limitation. Each member must be able to apply it to an unfamiliar component. Full answers, raw result analysis, live runbook.

Temperature observability implementation and evaluation

Group 15 • Final assessment preparation • 5 November 2026

Group 15 built a containerized temperature monitoring service for concurrent laboratory experiments. It consumes Kafka events, computes complete-sensor averages, preserves experimental history, exposes the required REST queries, and delivers transition notifications. The design separates ordered ingestion, stateless computation, authoritative persistence, public queries, and external notification delivery.

The latest suite passed 92 tests. Local execution of the original instructor tools passed changing HTTP targets within 100–200 requests/second and a separate fixed 200 target. An independent live-history workload passed at a 100-request/second peak but failed at 200, identifying a remaining capacity boundary on the shared two-vCPU deployment. These outcomes guide both our design defense and future work.

Requirements and domain interpretation

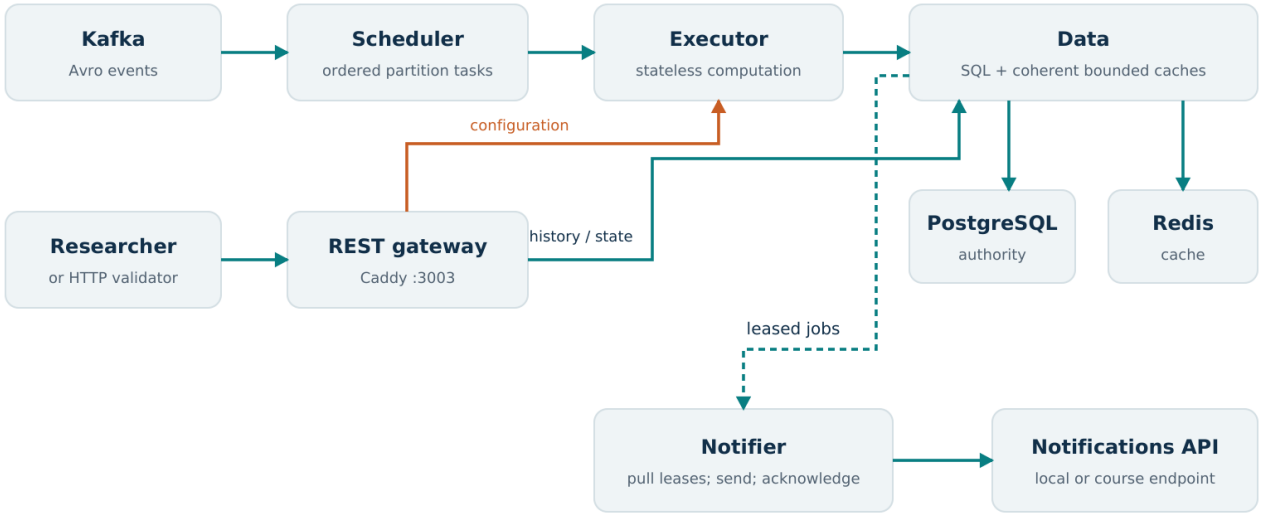
The system observes temperature; another system controls it. An experiment's temperature is the arithmetic mean of all allocated sensors for the same measurement identity. Combining the latest readings across different sampling instants could invent a temperature that was never measured, so incomplete groups wait and are persisted across executor replacement.

Configuration makes an experiment queryable. Stabilization triggers a Stabilized notification when the complete average enters the inclusive configured range. The experiment starts only on the researcher's start event. During carry-out, the first outside measurement of each excursion triggers OutOfRange; subsequent outside measurements remain in historical queries without generating another alert. Termination retains prior history. Stabilization aggregates can be persisted internally for processing and deduplication, but required history responses filter to carry-out.

The required outside endpoint returns all outside measurement points. A separate convenience endpoint derives excursion intervals. Thus three consecutive outside samples produce three outside-history points but one transition notification. Treating these counts as interchangeable would violate the interface contract.

Architecture and rationale

Application responsibilities and data flow



PostgreSQL owns durable state. Redis is disposable. The notifier pulls jobs from Data.

Kafka intake is implemented by a scheduler that decodes Avro records, maintains ordered partition queues, and dispatches bounded batches to the least-busy available executor. Local reservations prevent multiple assignments from relying on the same stale load observation. One active task per partition preserves order, assuming the producer keeps each experiment on one partition.

Executors load durable state, apply the pure state machine, and propose a commit. They do not own persistent experiments. Data owns the PostgreSQL transaction, query/cache behavior, and notification leases. Caddy exposes the public API, forwarding reads directly to Data and configuration to an executor. A separate notifier drains the durable outbox with bounded concurrency.

A modular monolith would reduce service hops and operational overhead on this small VM. Explicit services were chosen to match responsibilities with different ordering and scaling needs: computations can move between workers, history reads benefit from caches, and external delivery delays should not block intake. The measured replica results also show the cost: extra containers on the same CPUs do not create additional capacity.

PostgreSQL is authoritative because a single transaction can link state, history, outbox obligations, and processing progress. Redis is a disposable response cache. An additional bounded in-process snapshot cache accelerates small histories; larger histories retain the indexed SQL/Redis path. This keeps persistent correctness independent of either cache while limiting RAM growth.

Consistency and recovery

Data commits experiment versions, completed measurements, notification jobs, rejected input, and a partition checkpoint atomically. Kafka's next-read offset is acknowledged only after durable success. A crash before commit leaves work to recompute. A crash after commit but before acknowledgement can replay events, which the database checkpoint and measurement identities deduplicate.

Expected-version checks prevent silent lost updates. Batched writes retain deterministic lock order and verify every returned version; a conflict rolls back the complete transaction. An integration regression deliberately makes the middle of three proposed versions stale and verifies that none of them partially commits.

Cache correctness is also explicit. Histories and their version are read from one SQL statement. Metadata invalidation and active-writer generations prevent a delayed old writer from warming obsolete metadata

after a newer commit. A snapshot can serve a request only when it is sufficiently current. Cold readers share a fill, and cache eviction changes performance rather than deleting history.

Notifications use a transactional outbox and leases. This preserves the obligation through process loss, but it does not make an external HTTP side effect exactly once. A receiver may accept a notification before the sender loses the response. Retrying then needs receiver-side idempotency for unique acceptance. A fifteen-second lease can also exceed the ten-second deadline during failure recovery; durability and latency through outages are distinct properties.

Implementation evolution

Development began with contract and state-machine correctness: complete groups, phases, inclusive boundaries, duplicates, and replay. Integration exposed a producer compatibility detail: termination messages use a started-event Avro schema, while the Kafka `record_name` header correctly identifies termination. The decoder honors that header, with a regression test.

An early full instructor workload returned correct histories but had three late notifications and insufficient completed HTTP throughput. State writes were changed from sequential per-experiment database round trips to sorted, pipelined operations with all version checks preserved. Under the same configured October workload, maximum notification latency changed from 10.716 to 5.641 seconds and achieved HTTP throughput from 86.848 to 97.310/s. Random temperature paths differed, so these observations do not isolate a statistical causal effect.

After disk exhaustion and container/image cleanup, the retained PostgreSQL volume reopened with 2,326,784 measurements. The recovered, larger database revealed avoidable monitoring and read work. November changes added bounded versioned history snapshots, commit-time metadata warming with race fencing, indexed notification-status counts, and less frequent scans of historical monitoring totals. Pool wait and connection occupancy are now measured separately.

The local test controller itself was also expensive. Repeated Compose graph inspection consumed substantially more CPU than direct container inspection. The runner now caches container IDs and inspects both generators together. A separate raw-transport/uvloop experiment did not improve the independent boundary workload and was reverted rather than claimed as an optimization.

Restoring the demo consumer revealed 2,860,538 offsets of retained backlog in the initial broker snapshot. The scheduler gained per-partition high/low watermarks so one hot partition could not monopolize intake buffering. The existing consumer group caught up to the sampled broker positions without resetting offsets. Historical alerts from the outage remained late, and data no longer retained by Kafka cannot be reconstructed by this recovery.

Evaluation

The local host is an AWS t3a.medium in eu-west-1, with two vCPUs, approximately 3.754 GiB RAM, and 20 GiB provisioned disk. Application, databases, local Kafka, generators, and monitoring shared those resources. Both native tests used 100 main experiments, ten sensors per experiment, one-second samples, and a completed warmup before the main load.

Test	Notification result	HTTP and consistency result
Original tools, variable targets 100–200	703/703; maximum 2.884 s	Five observed target phases passed; 202 complete history comparisons
Original tools, fixed target 200/s	596/596; maximum 8.958 s	190.573 achieved/s; 202 complete history comparisons; narrow local pass
Independent live histories, 25/100/50 profile	1,626/1,626; maximum 6.189 s	All phases passed; peak achieved 97.926/s; no mismatches

Test	Notification result	HTTP and consistency result
Independent live histories, 50/200/100 profile	1,626/1,626; maximum 8.527 s	Failed: busy phase achieved 150.329/s and omitted some offered requests, despite zero request errors

The native variable targets actually observed were 148, 182, 181, 114, and 106. Our 95% throughput threshold is a local operational rule, not a percentage specified by the rubric. Notification acceptance is strict: missing, invalid, nonfinite, negative, or at-least-ten-second values fail.

The independent scheduler accounts for intended send time, scheduling delay, semaphore waiting, omitted offers, and phase drain time. Therefore a low error count cannot conceal overload. Native and independent runs have different query timing, histories, pacing, and notification receivers; their results are complementary rather than interchangeable.

The final suite contains 78 unit and 14 integration cases. Recovery testing preserved four history points and three alerts through partial-group persistence, replay, component restarts, notification 503s, Redis loss, and PostgreSQL restart with the Data process cache cleared. These are specific tested failure cases, not a claim of host-level high availability.

Capacity measurements preceded the final scheduler fairness revision. That revision passed the complete final suite and was exercised on the retained live backlog. A repeat capacity run on the exact final revision remains useful follow-up evidence.

Sources: native variable, native fixed, independent pass, independent failure, tests, recovery.

Scalability, cost, and operational limits

Sensor input rate is E×S/T for E experiments, S sensors, and sample period T. Aggregate rate is approximately E/T. At 100 experiments, ten sensors, and one second, these are 1,000 sensor events/s and 100 aggregates/s. With 16 assigned partitions and executor capacity eight, simultaneous partition tasks are bounded by min(16, 8K) for K executors. This bounds concurrency, not throughput.

Historical replica experiments favored one executor on the shared host. The bounded autoscaler remains available, but is not continuously active in the demo. Data currently assumes one process owns all writes; adding Data workers requires shared invalidation or removal of process-local metadata caching. More workers cannot fix a saturated database, a hot sequential partition, or a slow external receiver.

Cost is estimated as one VM's hourly price plus provisioned disk times its monthly unit price divided by 730. The recorded estimate is approximately $0.043211/hour using a 20 GiB gp3 assumption. Disk type is unconfirmed; CPU surplus credits, public IPv4, traffic, taxes, discounts, and course infrastructure are excluded. The saved price evidence supports an estimate, not an account bill.

Local Kafka/Prometheus retention and log limits reduce disposable growth; alerts identify low disk and deadline misses. Experiment history still grows with duration, and the application has no completed archival or off-host backup/restore program. External infrastructure-alert delivery is not configured. Separating services does not remove the single VM/disk failure domain.

Conclusion and next steps

The implementation combines explicit correctness boundaries with measured revisions. The strongest evidence is the original tools' passing workloads, independent validation that retains failures, and a code-level explanation of how each optimization preserves consistency. The remaining priority is sustained demo health and representative repeated tests, including longer histories, event-rate changes, skew, and external-path validation. Additional resources or a shared invalidation design should follow measured bottlenecks and the allowed deployment constraints.

The architecture report, performance record, and complete code map provide the detailed evidence. Individual contribution statements must be supplied by the group from actual work; preparation roles are

not authorship claims.

Code map and complete implementation references

Group 15 • Final assessment preparation • 5 November 2026

Use this map to locate the mechanism behind an exam answer. Each public file in the source manifest has a purpose below and its complete contents in the current code appendix. The inventory includes existing helpers and does not assign authorship to the group.

Trace one task through the code

`codec.decode_event` -> `scheduler.dispatch_partition` -> `executor.prepare_commit` -> `engine.ExperimentStateManager.apply` -> Data commit transaction -> scheduler Kafka commit. Then Data claim -> `notification.NotificationHandler.deliver` -> Data lease result. For a read, Caddy -> Data metadata/version -> bounded snapshot or SQL/Redis -> ordered JSON.

Changes and their provenance

The original implementation report includes the earlier code and complete original patch. The November report with code includes every incremental application, runner, test and deployment change from that session, with before/after hashes. Its baseline includes pre-existing uncommitted work; git HEAD alone is not the full development baseline.

This assessment-pack work changes documentation and its build/verification helpers. It does not alter the application, instructor Rust logic, test thresholds or live deployment. Read the performance chapter for which changes have isolated regressions, before/after observations, or only mechanism-level explanations.

root configuration

File	Purpose and reason
.dockerignore	Exclude local secrets, caches and irrelevant artifacts from the application build context.
.gitignore	Keep private runtime configuration, credentials and generated test state out of version control.
Makefile	Documented operator/test entry points, including variable tests and generator-only stopping.
compose.course-test.yml	Original instructor binaries, local truth/receiver, variable-generator settings and local panel wiring.
compose.demo.yml	External topic/group, mTLS mounts, known hostname exception and course notification destination.
compose.local.yml	Independent local Kafka/sink wiring and bounded disposable broker retention.
compose.monitoring.yml	Operational Prometheus/Grafana/node exporter, provisioning and retention limits.
compose.yml	Core service boundaries, health/dependency wiring, durable volumes, limits and capped logs.
monitoring.env.example	Public tunable template, including allocated storage; real values live in an excluded private file.
pyproject.toml	Python package/dependency/tool/test-marker configuration.
requirements.lock	Pinned application dependency set for repeatable builds.

deploy

File	Purpose and reason
deploy/Caddyfile	Public-only route forwarding and gateway behavior; internal mutations are not exposed.
deploy/CourseTools.Dockerfile	Build unmodified instructor Rust binaries for independent validation.
deploy/CourseTools.Dockerfile.dockerignore	Restrict the instructor-tool build context; avoid including private/generated state.
deploy/Dockerfile	Application and gateway build stages; repeatable Python runtime and OpenAPI artifacts.
deploy/alerts.yml	Nine validated rules covering service/jobs/input, individual deadlines, pool wait and disk pressure.
deploy/course/prometheus.yml	Scrape local instructor metrics and explicit run status.
deploy/course/provisioning/dashboards/course.yml	Provision local course dashboard location.
deploy/course/provisioning/datasources/course.yml	Provision local course Prometheus and truth PostgreSQL datasources.
deploy/course/schema.sql	Separate original-validator truth/notification tables; not the application authority.
deploy/docs.html	Public API documentation page served by the gateway.
deploy/grafana/dashboards/temperature.json	Operational dashboard, including pool, cache, disk, costs and individual deadline misses.
deploy/grafana/provisioning/dashboards/default.yml	Provision operational dashboard location.
deploy/grafana/provisioning/datasources/prometheus.yml	Provision operational Prometheus datasource.
deploy/prometheus.yml	Operational service/node scrape targets and alert rule loading.

loadtests

File	Purpose and reason
loadtests/__init__.py	Package marker; no standalone service behavior.
loadtests/course.py	Original Rust self-test orchestration, ground-truth checks, variable phase evidence, panels and source/image build reuse.
loadtests/http_profile.py	Independent intended-send scheduling with bounded pending work, omitted-offer detection and phase drain accounting.
loadtests/matrix.py	Repeat deterministic profiles across executor/polling settings and save sizing evidence.
loadtests/notification_sink.py	Local independent-test receiver and fault injection; distinct from the original cryptographic validator.
loadtests/recovery.py	Exercise persisted partial groups, logical replay, delivery failures, component restarts and cold-cache SQL recovery.
loadtests/stress.py	Independent end-to-end workload driver, complete-history/alert validation, strict deadlines and phase/telemetry results.
loadtests/telemetry.py	Sample host CPU/disk, queue and Data operation counters; diagnostics include cohosted test tools.
loadtests/validation.py	Strict below-ten deadline predicate and conservative settled native-variable phase validation.
loadtests/workload.py	Build deterministic sensor/event truth, serialize supplied Avro and calculate observed distributions.

scripts

File	Purpose and reason
scripts/build.sh	Existing course repository build helper; inclusion does not claim new authorship.
scripts/check-course-kafka.py	Read-only external mTLS preflight using a separate temporary consumer group; never publishes or commits live offsets.
scripts/generate-group-tokens.sh	Existing token-generation helper; generated credentials are excluded.
scripts/prepare-monitoring-credentials.py	Prepare the narrow mounted credential files without printing their private values.
scripts/push.sh	Existing course image-push helper; not executed to prepare this pack.
scripts/save_env.sh	Existing environment helper; runtime secret files are excluded from the report.
scripts/setup_env.sh	Existing development environment helper; inclusion does not claim new authorship.

temperature_observability

File	Purpose and reason
temperature_observability/__init__.py	Package marker; no standalone service behavior.
temperature_observability/api.py	Development REST/OpenAPI contract; deployed Caddy routes implement the public path.
temperature_observability/autoscale.py	Opt-in bounded host policy with busy/idle hysteresis and dry-run; not continuously active in demo.
temperature_observability/codec.py	Avro object-container decoding and authoritative record_name header; returns explicit invalid envelopes.
temperature_observability/common.py	Shared metrics, asynchronous pooled transport, logging and supervised service tasks; transport tests guard compatibility.
temperature_observability/config.py	Validated service limits, Kafka/TLS options and one-VM plus provisioned-storage cost model.
temperature_observability/data.py	Authoritative atomic commit/load, version checks, query caches, writer fencing, outbox leases and diagnostic metrics.
temperature_observability/discovery.py	Discover executor addresses/load and reserve least-busy capacity without relying solely on stale observations.
temperature_observability/engine.py	Pure domain state machine: complete groups, ordered drain, phases, boundaries, deduplication and transition jobs.
temperature_observability/executor.py	Load snapshots, compute commit proposals, retry optimistic conflicts, enforce capacity; no authoritative worker-local state.
temperature_observability/healthcheck.py	Container probe entry point; readiness is distinct from business correctness under workload.
temperature_observability/history.py	Bounded versioned small-history snapshots, inclusive binary-search selection and interval rendering.
temperature_observability/models.py	Typed events, states, task lookups, commit proposals, partial groups, measurements and notification payloads.
temperature_observability/notification.py	Leased durable-job dispatch, token read, idempotency key, timeout/retry and individual deadline accounting.
temperature_observability/scheduler.py	Ordered partition dispatch, durable-before-Kafka acknowledgement, global/per-partition backpressure and status.
temperature_observability/schema.sql	Application tables/constraints and query/outbox indexes; PostgreSQL is the source of truth.

tests

File	Purpose and reason
tests/test_codec.py	Avro decoding, termination header compatibility and invalid-message behavior.
tests/test_cost.py	Provisioned-storage billing estimate and incomplete-configuration behavior.
tests/test_course_build_cache.py	Source/image fingerprint invalidation, including schemas and SQL metadata while excluding credentials.
tests/test_data_integration.py	Real PostgreSQL atomicity/version rollback, leases, replay, cache coherence, delayed writers and large-history fallback.
tests/test_engine.py	Domain means, lifecycle, allocated sensors, incomplete groups, duplicates and excursion semantics.
tests/test_history.py	Inclusive bounds, duplicate timestamps, version rejection, interval closure and entry/point eviction limits.
tests/test_load_validation.py	Reject misleading aggregate/phase results, omitted offered work and exactly-ten/invalid latency observations.
tests/test_notification.py	Delivery outcomes, retries/lease acknowledgement and strict deadline miss accounting.
tests/test_public_api_integration.py	Required public routes and query contract, including internal-route protection.
tests/test_scheduler.py	Ordering, reservation/capacity, backpressure and reassignment behavior; scaling policy boundaries.
tests/test_service_client.py	Shared HTTP transport response/error compatibility and compressed responses.

Documentation tooling

prepare_sources.py verifies the source snapshot and generates this map and full code appendix. build_reports.py adapts the earlier report renderer, reads saved JSON, draws figures and builds PDFs. verify_reports.py checks numeric claims, local links, PDF content/bounds, source identity and private-value exclusion, then creates a manifest and portable ZIP. These tools never start workloads or change Docker services.

The 69-file public snapshot does not include every unchanged Rust source in the assignment. Those remain the instructor repository; the local runner compiles them unchanged. The portable pack is for reading and review, not a replacement for the full deployment checkout.